⚠ **Exemple de démonstration :** Application et données fictives. Résultats illustratifs générés dans un environnement de test contrôlé (sandbox). Ce document ne constitue pas un audit de l'application d'un client tiers.

Vibe-Audit · Rapport de Sécurité Express

Audit passif & remédiation IA assistée · Échantillon commercial démonstrateur

Cible : **LeadFlow Sandbox (Démo fictive)**
Date de l'audit : **13 Octobre 2026**
Auditeur : **Kreesten Agboton**
Cadre légal : **Environnement de test contrôlé**

RÉPARTITION DES VULNÉRABILITÉS CONFIRMÉES (TOTAL : 4)

2 Élevées · 1 Moyenne · 1 Faible

2 Élevées (50%) | 1 Moyenne (25%) | 1 Faible (25%)

Contrôles examinés sans anomalie : 4

SYNTHÈSE EXÉCUTIVE DÉCIDEUR

L'audit passif de l'environnement **LeadFlow Sandbox** a mis en évidence deux vulnérabilités d'importance élevée permettant l'extraction non authentifiée de données clients et l'exposition d'une clé d'intégration tierce.

Aucune intrusion active n'a été réalisée. La gestion des sessions utilisateurs (JWT) est robuste. *Les composants de paiement Stripe n'ont pas été évalués dans cette mission (hors périmètre).* Les correctifs peuvent être déployés en moins d'une heure à l'aide des prompts fournis.

COUVERTURE DU PÉRIMÈTRE D'ÉVALUATION

Composant	Méthode d'inspection	Statut
Supabase RLS (Orders)	Recherche passive REST API	Constat Élevé
Bundles JS / Clés API	Inspection statique AST frontend	Constat Élevé
Gestion des Sessions Auth	Vérification JWT & cookies HTTP	Examiné conforme
Infrastructure Stripe	Non évalué dans cette mission	Hors périmètre

3 ACTIONS PRIORITAIRES IMMÉDIATES

1. **Activer et configurer la politique RLS sur la table des commandes (SEC-01)**
   Bloque l'accès public immédiat aux emails et montants des transactions (Détail page 2).
2. **Déplacer la clé secrète Resend vers une Edge Function serveur (SEC-02)**
   La clé API présente dans le build client permettait l'usurpation des envois d'emails transactionnels.
3. **Durcir les en-têtes HTTP de sécurité (SEC-03)**
   Ajout des en-têtes Content-Security-Policy et X-Frame-Options contre le clickjacking.

Kit de remédiation : Chaque constat en page 2 est accompagné d'un prompt formulé pour les assistants de code (Lovable, Bolt, Cursor) avec ses critères de validation technique.

GRAVITÉ : ÉLEVÉ | Constat confirmé

SEC-01 : Absence de Row Level Security (RLS) sur la table des commandes

`supabase/schema:orders`

1. IMPACT MÉTIER & CONSÉQUENCE

Fuite de données transactionnelles clients
La table `orders` est accessible en lecture directe sans filtrage d'utilisateur. Tout visiteur non authentifié disposant de la clé publique anon peut extraire l'historique complet des commandes, adresses emails et montants encaissés.

2. PREUVE TECHNIQUE REPRODUCTIBLE

CAPTURE VÉRIFIÉE : 13/10/2026 16:42 UTC — GET REST API · 200 OK

```
# Requête passive curl via la clé publique anon exposée côté client
curl -i -s "https://sandbox.vibe-audit.io/rest/v1/orders?
select=id,customer_email,amount&limit=2" \
  -H "apikey: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.***"

# En-tête HTTP certifiant le nombre total d'enregistrements exposés :
HTTP/1.1 200 OK
Content-Range: 0-1/142
[
  { "id": "ord_812", "customer_email": "m.***@gmail.com", "amount": 4900
},
  { "id": "ord_813", "customer_email": "agence.***@pro.fr", "amount":
12900 }
]
# Constat formel : 142 lignes accessibles publiquement sans session
active
```

3. REMÉDIATION IMMÉDIATE (PROMPT & SQL)

Correction proposée

Cible : Lovable / Bolt / Cursor / Supabase SQL — Prompt prêt à coller

```
-- Hypothèse : la table 'orders' possède une colonne 'user_id' liée à
auth.users.id
Dans Supabase, active la politique Row Level Security (RLS) sur la table
'orders' :

ALTER TABLE orders ENABLE ROW LEVEL SECURITY;

-- 1. Isolation stricte en lecture pour le propriétaire
CREATE POLICY "Lecture réservée au propriétaire"
ON orders FOR SELECT
USING (auth.uid() = user_id);

-- 2. Isolation stricte en insertion
CREATE POLICY "Insertion réservée au propriétaire"
ON orders FOR INSERT
WITH CHECK (auth.uid() = user_id);

-- 3. Protection en mise à jour (évite la modification d'ordres tiers)
CREATE POLICY "Modification réservée au propriétaire"
ON orders FOR UPDATE
USING (auth.uid() = user_id);
```

4. Critères de Validation Post-Déploiement

- ◯ **Test 1 (Après correction) :** Rejouer la commande curl avec la clé anon. Le serveur doit renvoyer un tableau vide `[]` ou un code HTTP 401/403.
- ◯ **Test 2 (Session utilisateur) :** Se connecter avec un compte client : vérifier que ses commandes personnelles restent visibles sans anomalie.
- ◯ **Re-contrôle J+14 :** Prestation éligible au protocole de vérification gratuite après mise en production réelle des correctifs.